

ECON 636 600:MACROECONOMIC THEORY I

Assignment 1 (Solution)

1. (50 points) Derive the Euler equation

$$u'[c_t] = \beta u'[c_{t+1}] f'(k_{t+1}); t = 0, \dots, T-1$$

by applying the Kuhn-Tucker condition to the following problem,

$$\begin{aligned} \max \quad & \sum_{t=0}^T \beta^t u(c_t), \\ \text{s.t.} \quad & c_t + k_{t+1} \leq f(k_t), t = 0, \dots, T \\ & -k_{T+1} \leq 0. \end{aligned}$$

Note that I've deleted all the slack constraints ($c_t \geq 0, t = 0, 1, \dots, T$, and $k_t \geq 0, t = 1, 2, \dots, T$) in the above problem, which simplifies your derivation. (Even if I include these constraints in the above, the multipliers for these constraints would be zero because they do not bind.)

The Lagrangian is

$$L = - \sum_{t=0}^T \beta^t u(c_t) + \sum_{t=0}^T \lambda_t (c_t + k_{t+1} - f(k_t)),$$

with first order conditions

$$\frac{\partial L}{\partial c_t} = -\beta^t u'(c_t) + \lambda_t = 0, \tag{1}$$

$$\frac{\partial L}{\partial k_{t+1}} = \lambda_t - \lambda_{t+1} f'(k_{t+1}) = 0. \tag{2}$$

Substituting (1) into (2), we have

$$\beta^t u'(c_t) = \beta^{t+1} u'(c_{t+1}) f'(k_{t+1}). \tag{3}$$

2. (50 points) Consider an economy with two price-taking firms. Firm 1 has a production technology $f(k_1, n_1) = k_1^{1/3} n_1^{2/3} + k_1$, while firm 2 has $g(k_2, n_2) = k_2^{2/3} n_2^{1/3} + n_2$. Suppose the total supply of capital is 5 ($k_1 + k_2 = 5$), and the total supply of labor is 3 ($n_1 + n_2 = 3$).
- (a) Formulate the equilibrium condition as a system of non-linear equations. (15 points)
- (b) Use newton method to find the equilibrium allocations and price levels. (35 points) (The system of equations should be modified before Newton's method is applicable.)

An equilibrium is prices (W, R) and allocations (k_1, n_1, k_2, n_2) such that each firm maximizes its profit and markets clear. In other words,

(a) (k_1, n_1) solves $\max k_1^{1/3} n_1^{2/3} + k_1 - Rk_1 - Wn_1$.

(b) (k_2, n_2) solves $\max k_2^{2/3} n_2^{1/3} + n_2 - Rk_2 - Wn_2$.

(c) $k_1 + k_2 = 5$.

(d) $n_1 + n_2 = 3$.

Approach 1: The conditions for firm's maximization problem are

$$\begin{aligned} \frac{1}{3} k_1^{-\frac{2}{3}} n_1^{\frac{2}{3}} + 1.0 &= R = \frac{2}{3} k_2^{\frac{-1}{3}} n_2^{\frac{1}{3}}, \\ \frac{2}{3} k_1^{\frac{1}{3}} n_1^{\frac{-1}{3}} &= W = 1.0 + \frac{1}{3} k_2^{\frac{2}{3}} n_2^{\frac{-2}{3}}, \end{aligned}$$

or

$$\begin{aligned} \frac{1}{3} k_1^{-\frac{2}{3}} n_1^{\frac{2}{3}} + 1.0 - \frac{2}{3} k_2^{\frac{-1}{3}} n_2^{\frac{1}{3}} &= 0, \\ \frac{2}{3} k_1^{\frac{1}{3}} n_1^{\frac{-1}{3}} - 1.0 - \frac{1}{3} k_2^{\frac{2}{3}} n_2^{\frac{-2}{3}} &= 0, \end{aligned}$$

or

$$\begin{aligned}\frac{1}{3}k_1^{-\frac{2}{3}}n_1^{\frac{2}{3}} + 1.0 - \frac{2}{3}(5 - k_1)^{-\frac{1}{3}}(3 - n_1)^{\frac{1}{3}} &= 0, \\ \frac{2}{3}k_1^{\frac{1}{3}}n_1^{-\frac{1}{3}} - 1.0 - \frac{1}{3}(5 - k_1)^{\frac{2}{3}}(3 - n_1)^{-\frac{2}{3}} &= 0.\end{aligned}$$

To solve the above system (two equations and two unknowns) by newton's method, compute the jacobian matrix as

$$\begin{aligned}jac(1,1) &= -\frac{2}{9}k_1^{-\frac{5}{3}}n_1^{\frac{2}{3}} - \frac{2}{9}(5 - k_1)^{-\frac{4}{3}}(3 - n_1)^{\frac{1}{3}} \\ jac(1,2) &= \frac{2}{9}k_1^{-\frac{2}{3}}n_1^{-\frac{1}{3}} + \frac{2}{9}(5 - k_1)^{-\frac{1}{3}}(3 - n_1)^{-\frac{2}{3}} \\ jac(2,1) &= \frac{2}{9}k_1^{-\frac{2}{3}}n_1^{-\frac{1}{3}} + \frac{2}{9}(5 - k_1)^{-\frac{1}{3}}(3 - n_1)^{-\frac{2}{3}} \\ jac(2,2) &= -\frac{2}{9}k_1^{\frac{1}{3}}n_1^{-\frac{4}{3}} - \frac{2}{9}(5 - k_1)^{\frac{2}{3}}(3 - n_1)^{-\frac{5}{3}}\end{aligned}$$

However, the newton algorithm following this approach works only if the initial guess is very close to the true solution. Otherwise, the updates might hit the region where $k_1 < 0$, or $n_1 < 0$, or $k_1 > 5$, or $n_1 > 3$, and collapse. There is no easy solution to the “collapse” problem, unless we rewrite the equations, as in approach 2 and approach 3 below.

Approach 2: Define $x_1 = (\frac{k_1}{n_1})^{1/3}$, $x_2 = (\frac{k_2}{n_2})^{1/3}$. Then we have

$$\begin{aligned}\frac{1}{3}(x_1)^{-2} + 1 - \frac{2}{3}(x_2)^{-1} &= 0, \\ \frac{2}{3}(x_1) - \frac{1}{3}(x_2)^2 - 1 &= 0.\end{aligned}$$

We have two equations and two unknowns. To use the newton method, we need to rewrite the above equations to avoid singularity. (we should avoid x^{-1} , because when x is small, the slope converges to ∞) So the equations are rewritten as

$$\begin{aligned}\frac{1}{3}(x_2) + (x_1)^2(x_2) - \frac{2}{3}(x_1)^2 &= 0, \\ \frac{2}{3}(x_1) - \frac{1}{3}(x_2)^2 - 1 &= 0.\end{aligned}$$

The jacobian is

$$\begin{aligned} \text{jac}(1, 1 : 2) &= (2x_1x_2 - \frac{4}{3}x_1, \frac{1}{3} + (x_1)^2), \\ \text{jac}(2, 1 : 2) &= (\frac{2}{3}, \frac{-2}{3}x_2). \end{aligned}$$

Execute the code “newton.f90”, we get $x_1 = 1.678, x_2 = 0.596$, then $\frac{k_1}{n_1} = 4.725$ and $\frac{k_2}{n_2} = 0.212$. Thus we have $n_1 = 0.967, n_2 = 3 - n_1 = 2.033$. $k_1 = 0.967 * \frac{k_1}{n_1} = 4.569, k_2 = 2.033 * \frac{k_2}{n_2} = 0.431$.

```
program newton
```

```
! this code solves the question in homework 1
```

```
real:: x(2),d(2),fold(2),fval(2),jac(2,2),invjac(2,2),u(2)
```

```
integer:: i
```

```
i=0
```

```
x=(/0.5,0.5/)
```

```
d=(/1.0,1.0/)
```

```
CALL sub(x)
```

```
invjac(1,:)=(/jac(2,2),-jac(1,2)/)
```

```
invjac(2,:)=(/-jac(2,1),jac(1,1)/)
```

```
invjac=invjac/(jac(1,1)*jac(2,2)-jac(1,2)*jac(2,1))
```

```
do while ((MAXVAL(ABS(d))>1e-6).and.(MAXVAL(ABS(fval))>1e-6).and.i<1000)
```

```

i=i+1

d=-MATMUL(invjac,fval)

x=x+d

fold=fval

CALL sub(x)

invjac(1,:)=(/jac(2,2),-jac(1,2)/)

invjac(2,:)=(/-jac(2,1),jac(1,1)/)

invjac=invjac/(jac(1,1)*jac(2,2)-jac(1,2)*jac(2,1))

print *, 'iteration', i, 'x:', x, 'updates:', d

enddo

print *, fval

```

CONTAINS

```

SUBROUTINE sub(x)

real:: x(2)

fval=(/1.0/3.0*x(2)+x(1)**2*x(2)-2./3.*x(1)**2,&
      2./3.*x(1)-1./3.*x(2)**2-1./)

jac(1,1)=2.*x(1)*x(2)-4./3.*x(1)

jac(1,2)=1./3.+x(1)**2

jac(2,1)=2./3.

jac(2,2)=-2./3.*x(2)

```

END SUBROUTINE sub

end program newton

Approach 3: Define $x_1 = \log(k_1)$, $x_2 = \log(n_1)$, $x_3 = \log(k_2)$, $x_4 = \log(n_2)$. Then we have

$$\begin{aligned}\frac{1}{3}e^{-\frac{2}{3}x_1+\frac{2}{3}x_2} + 1.0 - \frac{2}{3}e^{\frac{-1}{3}x_3+\frac{1}{3}x_4} &= 0, \\ \frac{2}{3}e^{\frac{1}{3}x_1+\frac{-1}{3}x_2} - 1.0 - \frac{1}{3}e^{\frac{2}{3}x_3+\frac{-2}{3}x_4} &= 0 \\ e^{x_1} + e^{x_3} - 5 &= 0 \\ e^{x_2} + e^{x_4} - 3 &= 0.\end{aligned}$$

The jacobian is

$$\begin{aligned}jac(1, 1 : 4) &= \left(-\frac{2}{9}e^{-\frac{2}{3}x_1+\frac{2}{3}x_2}, \frac{2}{9}e^{-\frac{2}{3}x_1+\frac{2}{3}x_2}, \frac{2}{9}e^{\frac{-1}{3}x_3+\frac{1}{3}x_4}, -\frac{2}{9}e^{\frac{-1}{3}x_3+\frac{1}{3}x_4}\right), \\ jac(2, 1 : 4) &= \left(\frac{2}{9}e^{\frac{1}{3}x_1+\frac{-1}{3}x_2}, -\frac{2}{9}e^{\frac{1}{3}x_1+\frac{-1}{3}x_2}, -\frac{2}{9}e^{\frac{2}{3}x_3+\frac{-2}{3}x_4}, \frac{2}{9}e^{\frac{2}{3}x_3+\frac{-2}{3}x_4}\right), \\ jac(3, 1 : 4) &= (e^{x_1}, 0, e^{x_3}, 0), \\ jac(4, 1 : 4) &= (0, e^{x_2}, 0, e^{x_4}).\end{aligned}$$

We need to compute the inverse of a 4-by-4 matrix. The details are in the file, “newton.f90”, in elearning. Approach 3 relies on more computations, but is conceptually simpler than approach 2.